

MICRO-HARVEST

AI-Powered Agricultural Logistics Platform

Project Documentation · Version 2.0 · May 2026
Admin Panel: <https://micro-harvest.web.app>
Built by Morning Star · Banda, Uttar Pradesh, India

Table of Contents

1	Project Overview
2	System Architecture
3	Demo Flow Diagram
4	AI Operational Reasoning Engine
5	Gemini Model Fallback Chain
6	Offline-First Resilience Architecture
7	Demo Highlight Moments
8	Elasticsearch Integration
9	Cloud Functions
10	Flutter Applications
11	Payment System
12	Conversational Grower Agent
13	Firestore Schema
14	Security & Access Control
15	Demo Credentials & Testing
16	Complete Tech Stack
17	Project Timeline
18	Known Limitations & Future Roadmap
19	Repository & Submission

1. Project Overview

1.1 What is Micro-Harvest?

Micro-Harvest is an AI-native operational logistics orchestration platform for agricultural supply chains in rural India. It connects three parties in a seamless three-sided marketplace: **Growers** (farmers with surplus crops), **Producers** (wineries and buyers), and **Transporters** (haulage drivers). The platform eliminates middlemen, reduces post-harvest losses, and ensures fair transparent pricing through automated payment splits and tamper-evident contract hashing. Every step — from voice-based listing creation to final payment settlement — is orchestrated autonomously by AI.

1.2 Problem Statement

Indian farmers lose an estimated **16% of their produce annually** due to supply chain inefficiencies. Rural growers lack tools to:

- Quickly list surplus crops and find buyers in their language
- Connect with nearby transporters on demand based on real location data
- Receive fair transparent payments without intermediaries taking large cuts
- Operate effectively in low-connectivity rural environments
- Get intelligent logistics decisions explained in plain language

1.3 Operational Impact

Outcome	How Micro-Harvest Delivers It
Reduced crop spoilage	Urgency-aware matching prioritises perishable crops — HOURS_24 listings trigger faster transporter alerts
Faster pickups	Elasticsearch geo-distance queries find the nearest available transporter within seconds
Weather-aware logistics	AI Reasoning Engine flags high rainfall probability and increases urgency score automatically
Eliminated middlemen	Direct grower-to-producer matching with transparent 80/15/5 payment split
Rural resilience	Offline-first Hive queue ensures listings are never lost even in zero-connectivity areas
Fair pricing	Producers see real-time price-per-ton and total value — no hidden fees

1.4 Hackathon Context

Detail	Value
Hackathon	Google Cloud Rapid Agent Hackathon 2026
Track	Elastic Track
Contest Period	May 5, 2026 – June 11, 2026
Prize Pool	\$10,000 (Elastic Track)
Project Start	May 17, 2026

Admin Panel	https://micro-harvest.web.app
Tech Stack	Flutter, Firebase, Elasticsearch, Gemini AI, Stripe

1.5 AI Agent Evolution

Phase	Capability	What Changed
Phase 1 — Extraction	Single-turn NLP extraction	Grower submits text → Gemini extracts crop, weight, price in one shot → listing created
Phase 2 — Conversation	Multi-turn conversational orchestration	Persistent context via Firestore, follow-up questions, voice I/O, context accumulation
Phase 3 — Intelligence	Operational reasoning + explainability	Urgency scoring, weather risk analysis, decision factors shown to grower, Gemini tool-calling for ES transporter matching

2. System Architecture

2.1 High-Level Architecture — 4-App Monorepo

App	Platform	Role	Key Technology
Grower App	Flutter Android	Farmers list surplus crops via AI agent	Gemini NLP, STT/TTS, Hive offline
Producer App	Flutter Android	Buyers discover and claim listings	Elastic geo-search, Conversational search
Transporter App	Flutter Android	Drivers accept hauls and confirm delivery	FCM, GPS, Gate 1/2 confirmation
Admin Panel	Flutter Web	Platform management and monitoring	Elastic Monitor, Realtime Firestore

2.2 Backend Architecture

Service	Provider	Purpose
Authentication	Firebase Auth (Phone OTP)	All 3 roles use phone OTP login
Database	Cloud Firestore	Listings, handoffs, users, conversations
Cloud Functions	Firebase Functions v2 (asia-south1)	14 callable and trigger functions
File Storage	Firebase Storage	Gate 1/2 confirmation photos
Push Notifications	Firebase Cloud Messaging (FCM)	Haul alerts, status updates
AI / NLP	Google Gemini 2.5 Flash (7-model fallback)	Listing extraction, conversational agent
Search & Matching	Elastic Cloud (asia-south1)	Geo-search, transporter matching, MCP
Payments	Stripe (Test Mode)	Automatic settlement on Gate 2 confirmation
Web Hosting	Firebase Hosting	Admin panel at micro-harvest.web.app
Offline Storage	Hive (Flutter)	Offline queue for grower app

2.3 Complete Data Flow

Step	Who	What Happens	Technology
1	Grower	Speaks listing in natural language	STT (speech_to_text)
2	Cloud Function	Gemini extracts crop, weight, price, urgency	Gemini 2.5 Flash (with 7-model fallback)
3	Cloud Function	Listing indexed to Elasticsearch	onListingCreated trigger

4	Cloud Function	Gemini tool-calls find_nearby_transporters via ES	Elastic MCP + Agent Builder
5	Grower App	Review card shown, grower confirms listing	Flutter BLoC
6	Producer App	Listing appears in Discover feed via ES geo-search	Elasticsearch geo_point
7	Cloud Function	onProducerClaim — ES queries nearby transporters	Elastic geo-distance query
8	Transporter App	FCM haul alert received, transporter accepts	Firebase FCM
9	Cloud Function	onTransporterAccept — handoff document created	Firestore transaction
10	Transporter App	Gate 1: photo + GPS at pickup location	Camera, Geolocator
11	Transporter App	Gate 2: photo + GPS at delivery location	Camera, Geolocator
12	Cloud Function	onGate2Confirm — Stripe Payment Intent created	Stripe API
13	All Apps	SETTLED status + FCM notifications to all 3 parties	FCM multicast
14	All Apps	Earnings/purchases shown with correct 80/15/5 split	Firestore real-time

3. Demo Flow Diagram

The diagram below illustrates the complete end-to-end demo flow across all four apps, the Google Cloud Agent Builder, and the Elastic MCP integration. Each step is colour-coded by app: green for Grower, blue for Firestore/Producer, purple for Agent Builder/Elastic, amber for Admin Panel.

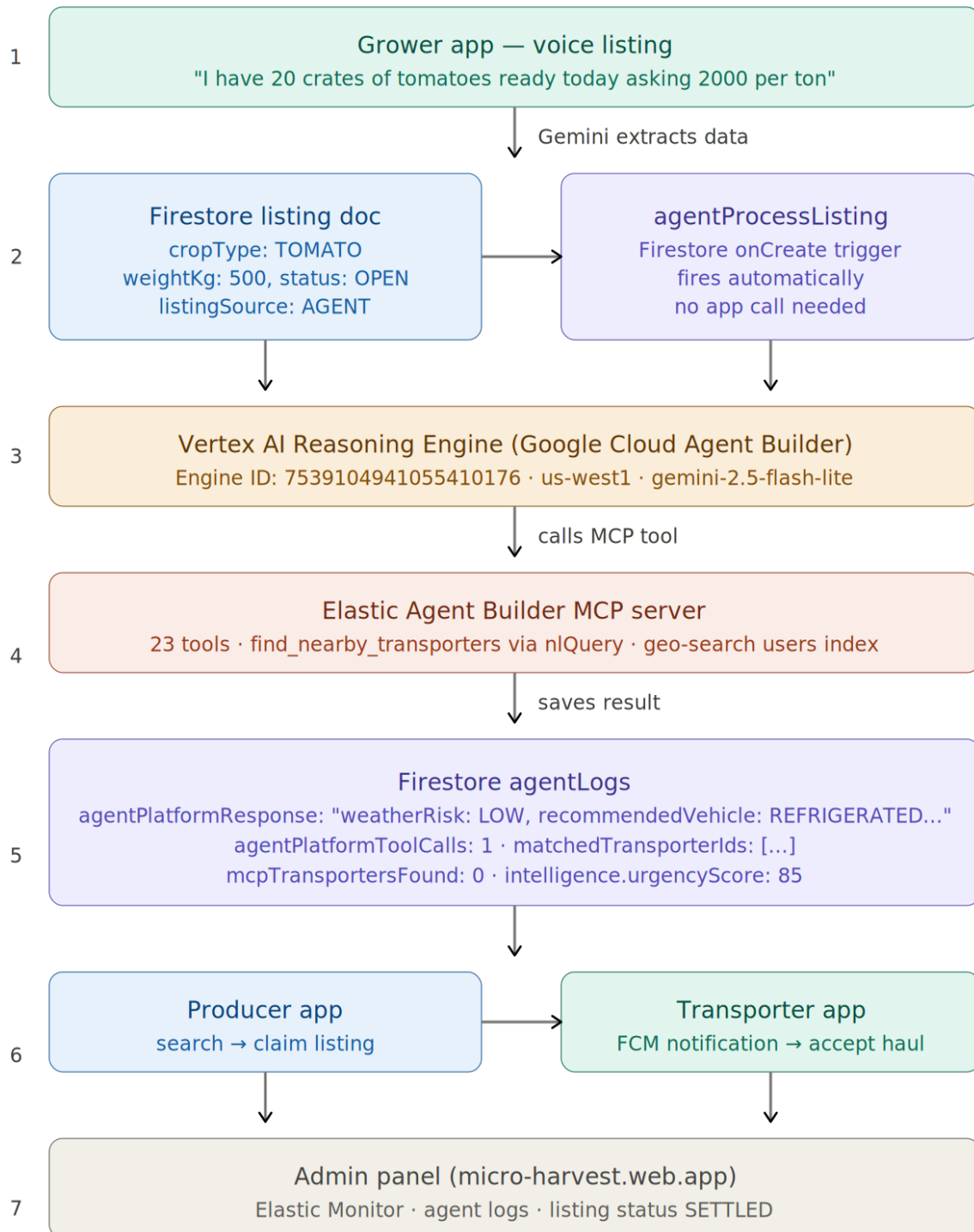


Figure 1 — Micro-Harvest demo flow: Grower voice listing → Gemini extraction → Elasticsearch indexing → Agent Builder tool-call → Producer discovery → Transporter FCM → Gate confirmation → Stripe settlement → Admin monitoring.

4. AI Operational Reasoning Engine

One of Micro-Harvest's key differentiators is its Explainable AI layer. After the grower confirms a listing, the Intelligence Service analyses the listing and produces a structured reasoning output visible to the grower in the AI Reasoning Card.

4.1 Reasoning Output Structure

IntelligenceService Output (stored in agentLog.reasoning)

urgencyScore:	number (0-100)	– overall urgency of the listing
weatherRisk:	HIGH MEDIUM LOW	– rainfall probability analysis
perishabilityRisk:	HIGH MEDIUM LOW	– based on crop type and perishTier
recommendedRadiusKm:	number	– suggested search radius
recommendedTransportType:	REFRIGERATED STANDARD	
decisionFactors:	string[]	– plain English reasons for each decision
historicalPriceAvg:	number null	– average price for crop type (future)

4.2 Urgency Scoring

Signal	Contribution	Example
perishTier	Primary factor	HOURS_24 → base score 70+
Weather risk	Boosts urgency	High rainfall → +15 points
Crop perishability	Secondary factor	Grapes (HIGH) → +10 points
Time since listing	Progressive boost	Older OPEN listings get higher urgency

4.3 Explainable AI Reasoning Card

Card Element	What It Shows	Example
Urgency Score	0-100 score with colour indicator	85 — HIGH urgency (red)
Weather Risk	Risk level with description	HIGH — Priority increased due to high rainfall probability (65%)
Perishability Risk	Crop-based risk assessment	HIGH — Grapes require same-day transport
Decision Factors	Plain English list of reasons	"Urgent delivery requested (HOURS_24)"
Recommended Radius	Suggested search area	150km — wider radius due to urgency

5. Gemini Model Fallback Chain

Micro-Harvest implements a **7-model automatic fallback chain** in the `geminiWithFallback` utility. If the primary model returns a retryable error (HTTP 429, 500, 502, 503, or 504), the system automatically retries with the next model in the chain without any manual intervention or user-visible failure.

■ Design principle: fast and cheap models first, high-capability models as last resort. This keeps latency low for the common case (simple extraction tasks) while guaranteeing availability under load.

5.1 Fallback Chain Order

Priority	Model ID	Role	Why This Position
1 — Primary	gemini-2.5-flash	Main model	Proven, fast, well-supported — handles 99% of requests
2	gemini-2.5-flash-lite	Fast fallback	Fastest and cheapest — ideal first fallback under rate limits
3	gemini-3-flash-preview	Next-gen flash	Next generation flash; slightly newer architecture
4	gemini-3.1-flash-lite-preview	Lite preview	Lightweight preview model — low latency
5	gemini-3.5-flash-lite	Newer lite	Newer lite generation — additional coverage
6	gemini-2.5-pro	High capability	Significantly more capable — reserved for complex reasoning
7 — Final	gemini-3.1-pro-preview	Maximum capability	Most capable model available — final safety net

5.2 Retryable vs Non-Retryable Errors

HTTP Status	Meaning	Action
429	Rate limit exceeded	■ Retry with next model
500	Internal server error	■ Retry with next model
502	Bad gateway	■ Retry with next model
503	Service unavailable	■ Retry with next model
504	Gateway timeout	■ Retry with next model
400	Bad request (prompt issue)	■ Fail immediately — model switch won't help
401	Auth error	■ Fail immediately — all models share same API key

5.3 `geminiWithFallback` Utility — Core Implementation

```
// functions/src/utls/geminiWithFallback.js
```

```

const FALLBACK_MODELS = [
  "gemini-2.5-flash",
  "gemini-2.5-flash-lite",
  "gemini-3-flash-preview",
  "gemini-3.1-flash-lite-preview",
  "gemini-3.5-flash-lite",
  "gemini-2.5-pro",
  "gemini-3.1-pro-preview",
];

const RETRYABLE_CODES = [429, 500, 502, 503, 504];

async function geminiWithFallback(prompt, options = {}) {
  const { systemInstruction, tools, generationConfig = {},
    startModel = "gemini-2.5-flash" } = options;

  const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
  const startIdx = FALLBACK_MODELS.indexOf(startModel);
  const chain = startIdx >= 0 ? FALLBACK_MODELS.slice(startIdx) : FALLBACK_MODELS;

  for (const modelName of chain) {
    try {
      const model = genAI.getGenerativeModel({
        model: modelName, systemInstruction, tools
      });
      const result = await model.generateContent({
        contents: [{ role: 'user', parts: [{ text: prompt }] }],
        generationConfig,
      });
      return { result, modelUsed: modelName }; // ← logged to agentLog
    } catch (error) {
      const code = error?.status ?? error?.httpErrorCode?.status;
      if (!RETRYABLE_CODES.includes(code)) throw error;
      // continue to next model
    }
  }
  throw new Error("All models exhausted");
}

```

5.4 Chat Variant for Multi-Turn Conversations

The `geminiChatWithFallback` function handles multi-turn conversations by rebuilding the chat session with full history on each fallback attempt. This is used by `processConversationTurn` to preserve conversation context even when switching models.

```

async function geminiChatWithFallback(history, newMessage, options = {}) {
  // Same chain logic as geminiWithFallback
  for (const modelName of chain) {
    try {
      const model = genAI.getGenerativeModel({ model: modelName, systemInstruction });
      const chat = model.startChat({ history, generationConfig });
      const result = await chat.sendMessage(newMessage);
    }
  }
}

```

```
        return { result, modelUsed: modelName };
    } catch (error) {
        if (!RETRYABLE_CODES.includes(error?.status)) throw error;
    }
}
}
```

5.5 Usage in Cloud Functions

```
// agentProcessListing – single-turn extraction
const { result, modelUsed } = await geminiWithFallback(prompt, {
  systemInstruction: "You are an agricultural logistics agent...",
  generationConfig: { temperature: 0.2 },
});
// modelUsed is stored in agentLog for observability

// processConversationTurn – multi-turn chat
const { result, modelUsed } = await geminiChatWithFallback(
  conversationHistory, userMessage, { systemInstruction }
);
```

■ **Observability tip:** `modelUsed` is written to every `agentLog` document. During the demo, open Firestore and show judges the field — it proves which model handled each request and demonstrates the fallback system is wired end-to-end, not just in code.

6. Offline-First Resilience Architecture

Micro-Harvest is designed for rural India where internet connectivity is unreliable. The grower app implements a complete offline-first architecture ensuring zero data loss even in dead zones.

Component	Technology	Purpose
Local Storage	Hive (Flutter)	Persist pending actions locally on device
Queue System	pendingActions Hive box	FIFO queue of unsent listing confirmations
Connectivity Detection	connectivity_plus	Monitor network status changes in real-time
Sync Engine	SyncEngine (custom)	Process queue automatically when internet returns
Sync Events	Dart StreamController	Broadcast sync completion to UI
Conversation Drafts	conversationDrafts Hive box	Persist partial conversations offline

6.1 Offline Architecture Overview

Step	State	What Happens
1	Offline detected	connectivity_plus returns ConnectivityResult.none
2	Grower confirms listing	AgentBloc checks connectivity BEFORE calling Cloud Function
3	Queued locally	Full extractedData + plotLocation saved to Hive pendingActions box
4	User notified	Orange snackbar: "Saved offline. Will sync when you reconnect."
5	Internet restored	SyncEngine.processQueue() fires automatically
6	Listing created	Direct Firestore write with listingSource: AGENT_OFFLINE
7	ES indexed	onListingCreated Firestore trigger fires → indexes to Elasticsearch
8	User notified	Green snackbar: "Your offline listing is now live!"

7. Demo Highlight Moments

These are the most impactful moments to demonstrate during the hackathon presentation — moments where the technology is visibly impressive to judges.

Moment	What to Show	Why It Impresses
Voice listing creation	Grower speaks "20 macro bins of Merlot today asking 4000 per ton" → listing created in 2 turns	Natural language → structured data → live marketplace in seconds
AI Reasoning Card	After listing, show urgency score 85, weather risk HIGH, decision factors	Explainable AI — system shows its reasoning transparently
Gemini tool-calling	Show agentLog in Firestore with matchedTransporterIds populated from ES geo-query	Proves real Elastic MCP integration, not just README mentions
Model fallback audit	Show modelUsed field in agentLog — confirm which Gemini model handled the request	Demonstrates production-grade resilience with 7-model fallback chain
Producer smart search	Type "urgent Merlot under 3000 per ton" → Gemini extracts → ES returns results	Natural language → Elastic query pipeline visible end-to-end
Automatic payment	Gate 2 confirmed → all 3 apps immediately show SETTLED with correct 80/15/5 split	Zero manual intervention — fully automated settlement
Offline sync	Turn off internet → confirm listing → turn on → green snackbar → listing in Firestore	Proves rural resilience — works in dead zones
Admin Elastic Monitor	Show cluster GREEN, listings indexed, transporter indexed, live search test	Enterprise-grade observability built into the platform
3-way FCM notification	Gate 2 triggers → all 3 phones receive "Delivery Complete" notification simultaneously	Real-time coordination across 3 parties in one action

8. Elasticsearch Integration

The Elastic integration is the core track requirement. Micro-Harvest uses Elasticsearch for three distinct purposes:

8.1 MCP Server & Agent Builder

Component	Value
MCP Endpoint	https://my-elasticsearch-project-d933e7.kb.asia-south1.gcp.elastic.cloud/api/agent_builder/mcp
Tool 1	search_crop_listings — geo-distance search on micro-harvest-listings index
Tool 2	find_nearby_transporters — geo-distance search on users index
Agent	Elastic AI Agent with 23 tools including 2 custom Micro-Harvest tools

8.2 Elasticsearch Indexes

micro-harvest-listings Index

Field	Type	Purpose
cropType	keyword	Filter by crop variety
weightKg	float	Weight of crop in kilograms
urgency	keyword	Perishability tier (HOURS_24, HOURS_48, DAYS_3)
status	keyword	OPEN, MATCHED, LOCKED, SETTLED
location	geo_point	GPS coordinates of grower plot
askingPricePerTon	float	Price per metric ton
growerId	keyword	Firebase UID of grower
createdAt	date	Listing creation timestamp

users Index

Field	Type	Purpose
uid	keyword	Firebase UID
role	keyword	TRANSPORTER only (producers/growers excluded)
availabilityStatus	keyword	AVAILABLE or UNAVAILABLE
location	geo_point	Transporter home base GPS
radiusMiles	float	Operating radius preference
fcmTokens	keyword	Push notification tokens
suspended	boolean	Account suspension flag

8.3 Real-Time Status Sync

Cloud Function	Trigger	ES Update
onProducerClaim	Producer claims listing	status → MATCHED
onTransporterAccept	Transporter accepts haul	status → LOCKED
onGate2Confirm	Gate 2 delivery confirmed	status → SETTLED
onListingCreated	New listing in Firestore	Full document indexed with geo_point
onUserSync	User document updated	Transporter profile synced to users index

9. Cloud Functions

14 Cloud Functions deployed to asia-south1 region:

Function	Type	Purpose
agentProcessListing	Callable	Gemini NLP extraction + ES tool-calling + listing creation
processConversationTurn	Callable	Multi-turn conversational agent with context persistence
onListingCreated	Firestore trigger	Index new listing to Elasticsearch
onProducerClaim	Firestore trigger	ES transporter geo-search + FCM haul alerts
onTransporterAccept	Callable	Create handoff document + lock listing + notify parties
onGate1Confirm	Callable	Record Gate 1 GPS + photo hash + update listing status
onGate2Confirm	Callable	Record Gate 2 + create Stripe Payment Intent + settle
onProducerSettle	Callable	Manual settlement trigger (backup to automatic)
onUserSync	Firestore trigger	Sync TRANSPORTER profiles to Elasticsearch users index
producerSearch	Callable	Gemini NLP → Elastic geo-search for producer app
getElasticStats	Callable	Return ES cluster health + index counts for admin panel
syncListingToElastic	Callable	Real-time status sync OPEN→SETTLED
retryTransporterNotification	Callable	FCM retry for missed haul alerts
onDisputeRaised	Firestore trigger	Dispute workflow initiation

10. Flutter Applications

10.1 Grower App (Flutter Android)

Screen	Description
Dashboard	Stats, recent listings, quick access
AI Agent	Multi-turn conversational listing creation with STT/TTS
Listings	All listings with status filters
Listing Detail	Status timeline, producer/transporter info, payment breakdown
Earnings	Payment history showing grower 80% share
Onboarding	Business name, GPS location, crop interests
Profile	Edit name, location, crop interests

- Natural language listing creation via Gemini AI
- Voice input (speech_to_text) + voice responses (flutter_tts)
- Multi-turn conversation with Firestore context persistence
- Container weight auto-calculation (MACRO_BIN=500kg, etc.)
- Offline queue via Hive — listings sync automatically on reconnect
- AI Reasoning Card showing urgency score, weather risk, perishability
- 7-model Gemini fallback chain — modelUsed logged to agentLog

10.2 Producer App (Flutter Android)

Screen	Description
Discover	Elasticsearch geo-search listing feed with filters
Smart Search	Conversational NLP search via Gemini + Elastic
Listing Detail	Full listing info with Claim button
Active Hauls	In-progress hauls with live transporter map
Cargo Confirm	Gate 2 delivery confirmation + payment summary
Purchases	Payment history with 80/15/5 split breakdown

- Discover tab powered by Elasticsearch geo-distance query
- Smart Search — speak or type queries, Gemini extracts params, Elastic returns results
- Live transporter tracking map on active hauls
- Real-time payment status (Pending Settlement → Payment Processed)

10.3 Transporter App (Flutter Android)

Screen	Description
Dashboard	Availability toggle, active haul, today's earnings
Haul Alert	Incoming haul with pickup/delivery locations + estimated earnings

Gate 1	Pickup confirmation — GPS + camera at grower plot
Gate 2	Delivery confirmation — GPS + camera at producer location
Hauls History	Past completed hauls
Earnings	Payment history showing transporter 15% fee

- Availability toggle syncs to Elasticsearch in real-time via onUserSync
- FCM deep-link navigation — notification tap opens HaulAlertScreen directly
- Gate 1/2 GPS + photo capture with SHA-256 image hash for tamper detection
- Automatic payment triggered on Gate 2 confirmation

10.4 Admin Panel (Flutter Web — <https://micro-harvest.web.app>)

Screen	Data Source	Features
Dashboard	Firestore aggregation	Total revenue, active listings, user counts, recent hauls
Listings	Firestore /listings	All listings with status badges, grower/producer names
Users	Firestore /users	All users by role with suspend/activate actions
Handoffs	Firestore /handoffs	Payment breakdowns, Gate 1/2 status, Stripe IDs
Disputes	Firestore /handoffs where disputeStatus=RAISED	Resolve/dismiss dispute actions
Analytics	Firestore aggregation	Revenue charts, haul volume, crop type distribution
Elastic Monitor	getElasticStats function	Cluster health, index counts, live search test

11. Payment System

Step	Action	Result
1	Transporter confirms Gate 2 delivery	onGate2Confirm Cloud Function fires
2	Function calculates total: askingPricePerTon × weightTons	e.g. ₹4000/ton × 10 tons = ₹40,000
3	3-way split calculated	Grower 80%, Transporter 15%, Platform 5%
4	Stripe Payment Intent created	amountPaise = totalINR × 100
5	Firestore handoff updated	payment.releasedAt set, stripePaymentId stored
6	Listing status → SETTLED	Removed from discovery feeds
7	FCM sent to all 3 parties	"Delivery Complete — Payment Processed"
8	Earnings visible in all apps	Grower: 80%, Transporter: 15%

11.2 Contract Hash

Each handoff document includes a SHA-256 contractHash computed from listing data at the time of transporter acceptance. This provides an immutable audit trail proving the terms were agreed upon before delivery.

12. Conversational Grower Agent

Component	Technology	Purpose
Speech-to-Text	speech_to_text Flutter package	Convert voice to text (en-US)
Text-to-Speech	flutter_tts Flutter package	Agent speaks responses
Conversation Memory	Firestore /conversations/{id}	Full context persists across turns
Context ID	UUID v4 generated locally	Same ID used for all turns in a session
NLP Extraction	Gemini 2.5 Flash (with 7-model fallback)	Extract listing fields from natural language
Model Fallback	geminiWithFallback utility	Auto-retry on 503/429 with next model in chain
Offline Queue	Hive pendingActions box	Queue listings when offline
Auto Sync	connectivity_plus + SyncEngine	Process queue when internet restored

12.2 Conversation Flow

Turn	Grower Says	Agent Does	Fields Extracted
1	"I have 20 macro bins of Merlot today"	Extracts all fields, asks for price	cropType, containerType, containerCount, weightKg (auto), perishTier (inferred)
2	"3000 per ton"	All fields complete → shows review card	askingPricePerTon
3	Grower taps Confirm	Confirmation dialog shown	—
4	Grower taps Yes, List It	Listing created in Firestore + ES indexed	—

13. Firestore Schema

Collection	Document ID	Written By	Read By
users	Firebase Auth UID	App onboarding + onUserSync	All apps + admin
listings	Auto-generated	agentProcessListing + grower app	All apps + admin
handoffs	Auto-generated	onTransporterAccept	Transporter + producer + grower
agentLogs	Auto-generated	agentProcessListing	Grower app + admin
conversations	UUID v4 (client-generated)	processConversationTurn	Cloud Functions only
disputes	Auto-generated	onDisputeRaise	Admin panel

14. Security & Access Control

Collection	Read Access	Write Access	Notes
users	Own document + Admin	Own document (onboarding + profile)	Role locked after onboarding
listings	Any authenticated user + Admin	Grower creates; producer claims (OPEN only)	Claim: only producerId + status fields
handoffs	Parties involved + Admin	Producer: disputeStatus only; else via Cloud Functions	Admin bypasses via admin SDK
agentLogs	Own growerId + Admin	Cloud Functions only	No direct client write
conversations	Cloud Functions only	Cloud Functions only	Context managed server-side
disputes	Admin only	Cloud Functions only	Admin resolves/dismisses

■ Gate Photo Security: Each gate confirmation photo is hashed with SHA-256 and the hash stored in the handoff document. This prevents photo substitution after submission and provides a tamper-evident audit trail.

15. Demo Credentials & Testing

App	Phone Number	OTP	Notes
Grower App	+16666666666	123456	Firebase test phone number
Producer App	+918888888888	123456	Firebase test phone number
Transporter App	+917777777777	123456	Firebase test phone number
Admin Panel	admin@microharvest.com	Admin@123	Email + password auth

15.2 Stripe Test Card

Field	Value
Card Number	4242 4242 4242 4242
Expiry	Any future date
CVV	Any 3 digits
ZIP	Any 5 digits
Result	Always succeeds in test mode

15.3 3-Minute Demo Video Script

Time	Action	What to Show
0:00–0:20	Open Grower App	Welcome message spoken by TTS agent
0:20–0:50	Speak listing	"I have 20 macro bins of Merlot today asking 4000 per ton" → agent asks for confirmation only
0:50–1:10	Confirm listing	Review card → confirmation dialog → listing created
1:10–1:20	Show agentLog in Firestore	modelUsed field + matchedTransporterIds from ES — proof of fallback chain + MCP
1:20–1:35	Switch to Producer App	Listing appears in Discover feed with correct price
1:35–1:50	Producer claims listing	onProducerClaim fires → transporter gets FCM
1:50–2:05	Transporter accepts haul	HaulAlertScreen with pickup/delivery locations + estimated earnings
2:05–2:20	Gate 1 + Gate 2	Photo + GPS confirmation at each gate
2:20–2:35	All 3 apps show SETTLED	Correct payment split: 80/15/5
2:35–2:50	Open Admin Panel	https://micro-harvest.web.app — dashboard + Elastic Monitor
2:50–3:00	Elastic Monitor	Cluster GREEN, listings indexed, transporter indexed

16. Complete Tech Stack

16.1 Frontend (Flutter)

Package	Version	Purpose
Flutter	3.x	Cross-platform mobile + web framework
flutter_bloc	^8.x	State management (BLoC pattern)
go_router	^13.x	Declarative routing
firebase_core / cloud_firestore / firebase_auth	^3–5.x	Firebase integration
cloud_functions / firebase_messaging / firebase_storage	^4–15.x	Functions, FCM, Storage
google_maps_flutter	^2.x	Maps for active hauls
geolocator	^13.x	GPS location
speech_to_text / flutter_tts	^6 / ^4.x	Voice I/O
hive / hive_flutter	^2.x	Offline local storage
connectivity_plus	^6.x	Network connectivity detection
flutter_stripe	^10.x	Stripe payment sheet
image_picker	^1.x	Camera for gate photos
uuid	^4.x	Conversation ID generation
fl_chart	^0.68	Analytics charts in admin

16.2 Backend (Cloud Functions)

Package	Purpose
firebase-functions/v2	Cloud Functions framework
firebase-admin	Firestore + FCM admin SDK
@google/generative-ai	Gemini API client (7-model fallback chain)
@elastic/elasticsearch	Elasticsearch client v8
stripe	Stripe payment processing
dotenv	Environment variable management

16.3 Infrastructure

Service	Plan/Tier	Region
Firebase (Blaze)	Pay-as-you-go	asia-south1 (Mumbai)

Elastic Cloud	Serverless	asia-south1 (GCP)
Stripe	Test Mode	Global
Firebase Hosting	Free tier	Global CDN
Google Maps Platform	Standard	Global

17. Project Timeline

■ Built in 6 days (May 17–23, 2026) for the Google Cloud Rapid Agent Hackathon 2026.

Date	Milestone
May 17	Project started — initial Flutter + Firebase setup
May 18	Elastic MCP integration + Agent Builder tools configured
May 18	agentProcessListing function with Gemini tool-calling deployed
May 18	micro-harvest-listings + users indexes created with geo_point mapping
May 18	All 3 apps — core flows built (auth, onboarding, listings)
May 18	Gate 1/2 confirmation with GPS + photo capture
May 19	Stripe payment integration + automatic settlement on Gate 2
May 19	Admin panel built and deployed to Firebase Hosting
May 19	Earnings screens for all 3 roles
May 20	Elasticsearch status sync (real-time OPEN→SETTLED)
May 20	onUserSync trigger — transporter profiles auto-synced to ES
May 21	Conversational agent with multi-turn memory + context persistence
May 21	ConversationId UUID fix — context no longer lost between turns
May 21	Offline Hive queue + auto-sync on reconnect
May 21	7-model Gemini fallback chain implemented (geminiWithFallback utility)
May 23	Producer conversational search (Gemini + Elastic)
May 23	perishTier normalisation + weightKg lock + price extraction fixes
June 11	Submission deadline — 2:00 PM PDT

18. Known Limitations & Future Roadmap

Area	Current State	Planned Fix
Payments	Stripe test mode only — no real payouts	Stripe Connect integration post-hackathon
Crops	Limited to wine grapes + Indian crops	Expand to grains, spices, dairy
GPS Check	Gate 1/2 GPS proximity not enforced (dev mode)	Re-enable 500m validation for production
Rating System	Hardcoded 5.0 rating	Build actual rating/review system
Offline Agent	Offline conversations not replayed to Gemini	Background conversation replay queue
Weather Intelligence	Rain risk hardcoded at 65%	Integrate OpenWeatherMap / IMD API
Market Pricing	historicalPriceAvg always null	Aggregate real pricing data from handoffs

18.2 Post-Hackathon Roadmap

- Stripe Connect for real 3-way payment splits to bank accounts
- Expand crop types beyond wine grapes (vegetables, grains, spices)
- Weather API integration for dynamic urgency scoring
- AI market price recommendations based on historical data
- Turn-by-turn navigation for transporters
- Dispute resolution workflow with admin mediation
- WhatsApp notification fallback for low-connectivity areas
- Multi-language support (Hindi, regional Indian languages)
- White-label version for other agricultural regions

19. Repository & Submission

Item	Value
Admin Panel URL	https://micro-harvest.web.app
Repository	https://github.com/ssurekumar01111-hue/micro-harvest
License	MIT License
Devpost Submission	https://rapid-agent.devpost.com
Track	Elastic Track
Demo Video	YouTube/Vimeo (3 minutes max)
Submission Deadline	June 11, 2026 @ 2:00 PM PDT

19.2 Repository Structure

```
micro-harvest/  
  ■■■ apps/  
    ■ ■■■ grower/      # Flutter Android – Grower app  
    ■ ■■■ producer/    # Flutter Android – Producer app  
    ■ ■■■ transporter/  # Flutter Android – Transporter app  
    ■ ■■■ admin/       # Flutter Web – Admin panel  
  ■■■ functions/  
    ■ ■■■ src/  
      ■ ■■■ agent/      # agentProcessListing, processConversationTurn  
      ■ ■■■ listings/   # onProducerClaim, onTransporterAccept, onListingCreated  
      ■ ■■■ handoffs/   # onGate1Confirm, onGate2Confirm  
      ■ ■■■ payments/   # onProducerSettle  
      ■ ■■■ search/     # producerSearch  
      ■ ■■■ users/      # onUserSync  
      ■ ■■■ utils/      # geminiWithFallback (7-model chain)  
  ■■■ firestore.rules  
  ■■■ firebase.json  
  ■■■ README.md
```